ESCUELA POLITÉCNICA NACIONAL
Facultad de Ingeniería de Software
Construcción y Evolución de Software
Taller 1

Integrantes:

- Shirley Maldonado
- Scarleth Jarrín

Tema:

¿Qué es Ingeniería Inversa? Técnicas, herramientas y futuro.

Objetivo:

Apropiar los conceptos fundamentales de la ingeniería inversa, analizar su evolución histórica y tecnológica, y aplicar estos conocimientos para hipotetizar el funcionamiento de una plataforma de software moderna de alto impacto, utilizando Inteligencia Artificial como asistente de investigación y generación de contenido.

Fase 1: Calentamiento y Contextualización

1. **Ingeniería Inversa: (En sus propias palabras, ¿qué significa "echarse para atrás" en el ciclo de vida?)**
   En el ciclo de vida del software, “echarse para atrás” se refiere a volver a etapas anteriores del desarrollo, pero partiendo de un software que ya está hecho. Es decir, en lugar de comenzar desde los requisitos y avanzar hasta el código, se hace el proceso al revés.
   Mediante la ingeniería inversa, se analiza el sistema ya implementado para entender cómo funciona, cómo está diseñado y qué objetivos cumple, especialmente cuando no existe documentación o esta no está actualizada. De esta manera, se puede recuperar información importante del sistema que se perdió con el tiempo.

2. **Mantenimiento de Software: (¿Qué tipos existen y por qué es necesaria la ingeniería inversa aquí?)**
   El mantenimiento de software son todas las actividades que se hacen después de que un sistema ya está funcionando, para corregir fallos, mejorar funciones o adaptarlo a nuevos cambios.
   Existen cuatro tipos principales de mantenimiento:

- Correctivo: se encarga de corregir errores que aparecen cuando el software ya está en uso.
- Adaptativo: se realiza cuando el sistema necesita adaptarse a cambios del entorno, como nuevos sistemas operativos o tecnologías.
- Perfectivo: busca mejorar el rendimiento del sistema o agregar nuevas funcionalidades.
- Preventivo: se enfoca en mejorar el código para evitar problemas futuros.

La ingeniería inversa es necesaria porque muchas veces el software es muy antiguo, está mal documentado o nadie sabe cómo funciona internamente. Entonces, antes de modificarlo, hay que analizarlo para entenderlo.

3. **Evolución del Software: (¿Cómo ayuda la ingeniería inversa a que un sistema no se vuelva obsoleto?)**

   La ingeniería inversa ayuda a que el software no se vuelva obsoleto porque permite comprender y mejorar sistemas antiguos, en lugar de desecharlos y crear uno nuevo desde cero.

   Gracias a la ingeniería inversa, es posible:

   - Analizar tecnologías viejas y planificar su actualización.
   - Mejorar la estructura del sistema.
   - Facilitar migraciones a nuevas plataformas.
   - Alargar la vida útil del software.

   De esta forma, el sistema puede seguir evolucionando y adaptándose a nuevas necesidades, evitando quedar obsoleto.

Fase 2: Investigación Asistida por IA

<u>ANTES</u>

La ingeniería inversa de software surge formalmente en los años 70 y 80, cuando las organizaciones comenzaron a enfrentar la necesidad de **comprender, mantener y extender sistemas desarrollados sin documentación clara**. Sin embargo, fue en los **años 90** cuando tomó fuerza debido a tres factores clave:

1. **Crisis del software y sistemas legacy**: muchas empresas dependían de aplicaciones críticas escritas en COBOL, C, ensamblador o entornos propietarios que ya no tenían soporte ni desarrolladores disponibles.
2. **Migración tecnológica**: el paso a arquitecturas cliente-servidor y a sistemas basados en la web obligó a analizar software antiguo para adaptarlo o integrarlo.
3. **Seguridad y análisis de malware**: comenzó a crecer el interés por desensamblar y estudiar software tanto para auditorías como para proteger infraestructuras.

Así, la ingeniería inversa se consolidó como una disciplina para **descomponer software existente**, entender su comportamiento interno y permitir mantenimiento, auditoría, interoperabilidad o migración.

Herramientas y técnicas usadas en los años 90 para sistemas legacy

En esa década, el análisis se hacía con herramientas mucho más rudimentarias que las actuales. Las principales eran:

1. Desensambladores (Disassemblers)

Convertían el código máquina en instrucciones en ensamblador para su análisis manual. Ejemplos típicos:

- **SoftICE**
- **IDA (Interactive Disassembler)** – primeras versiones
- **W32Dasm**
- **Turbo Debugger** (incluía desensamblador)

2. Depuradores (Debuggers)

Permitían ejecutar el programa paso a paso, inspeccionar registros, memoria y puntos de interrupción.
Herramientas clásicas:

- **GDB** (GNU Debugger)
- **SoftICE** (muy popular en Windows 95/98 y NT)
- **Turbo Debugger** de Borland (para DOS)

- **CodeView** de Microsoft

3. Hex editors

Utilizados para examinar archivos binarios y modificar bytes específicos. Ejemplos:

- **Norton Disk Editor**
- **Hex Workshop**
- **HIEW (Hackers View)** – muy usado para editar ejecutables

4. Analizadores de binarios y utilidades del sistema

Para inspeccionar encabezados, dependencias, símbolos y estructura del ejecutable:

- **objdump**, **nm**, **strings**
- **Dependency Walker** (finales de los 90)
- **COFF/PE viewers** tempranos

5. Ingeniería inversa de alto nivel

Aunque limitada, ya existían herramientas que intentaban reconstruir código fuente parcial:

- **Decompiler de Borland** (muy básico)
- **REC – Reverse Engineering Compiler**
- **IDA + plugins rudimentarios**

6. Técnicas comunes de la época

- Trazado de ejecución ("tracing") paso a paso
- Análisis de flujo de control manual
- Reconstrucción de estructuras de datos a partir de patrones de memoria
- Detección de cadenas y recursos incrustados
- Parcheo de instrucciones (NOP, JMP, CALL)
- Comparación binaria entre versiones para detectar cambios

- Revisión de volcados de memoria ("memory dumps")
- Retroingeniería de protocolos sin documentación mediante sniffers (como **tcpdump**)

AHORA

La ingeniería inversa hoy es muy diferente debido a la complejidad de las aplicaciones modernas.

Herramientas actuales

- **Descompiladores avanzados** (Ghidra, IDA Pro moderno).
- **Herramientas de análisis de tráfico cifrado** (HTTP Toolkit, Burp Suite).
- **Descompilación de APKs** (APKTool, jadx).
- **Herramientas para microservicios** (inspección de APIs REST, contenedores, logs).
- **Monitorización dinámica** de apps móviles y web mediante entornos virtuales.

Cambios tecnológicos importantes

- Uso masivo de **cloud computing** (AWS, GCP, Azure).
- Arquitecturas de **microservicios**: aplicaciones divididas en partes pequeñas.
- **Ofuscación de código**: técnicas para esconder la lógica interna.
- Mayor uso de **cifrado** en todo (HTTPS, TLS, cifrado en APIs y apps móviles).

¿Por qué ahora es más difícil?

- Más seguridad.
- Más capas entre el usuario y el servidor.
- Más lenguajes modernos y frameworks grandes.

MAÑANA

La Inteligencia Artificial está transformando la ingeniería inversa.

Usos actuales de la IA

- **Descompilación asistida por IA:** la IA interpreta código desarmado y lo explica con lenguaje natural.
- **Análisis automático de malware:** identifica patrones sospechosos.
- **Reconstrucción de arquitecturas:** la IA estima cómo podría funcionar un sistema complejo.
- **Detección de vulnerabilidades:** identifica fallos de seguridad sin revisar manualmente el código.

Futuro cercano

- IA que reconstruye diagramas completos a partir de binarios.
- Modelos capaces de traducir código antiguo a lenguajes modernos.
- Asistentes que guían el proceso de ingeniería inversa de punta a punta.

Fase 3: Caso de Estudio — WhatsApp (Sistema de Mensajería en Tiempo Real)

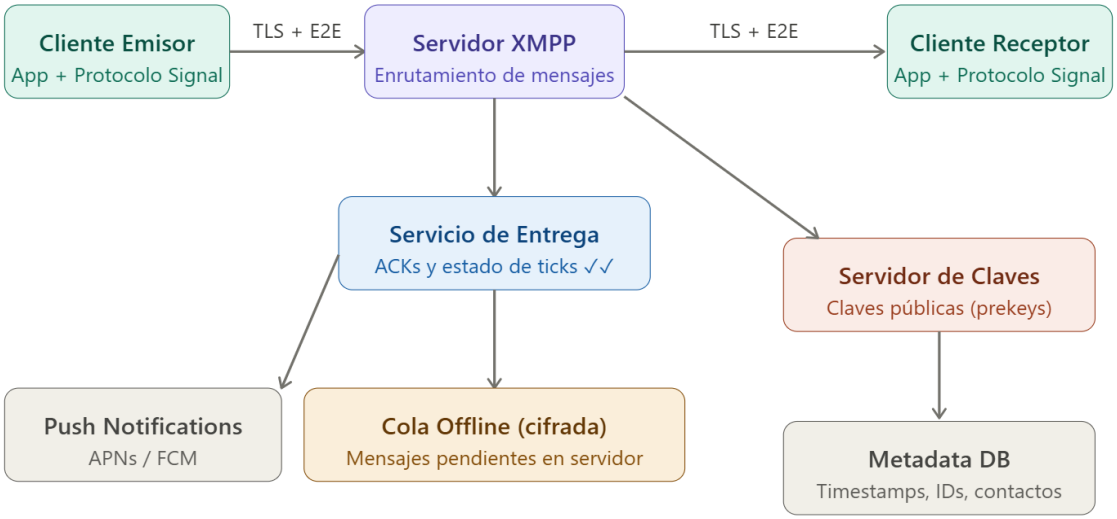
Objetivo

Funcionalidad: El sistema de mensajería en tiempo real con cifrado de extremo a extremo (End-to-End Encryption).

Técnicas de Ingeniería Inversa a aplicar

1. **API Sniffing (Análisis de tráfico de red)** Con herramientas como Wireshark o mitmproxy se intercepta el tráfico entre la app y los servidores. Aunque el contenido está cifrado, se puede observar a qué IPs conecta, con qué frecuencia, qué tamaño
2. tienen los paquetes, y qué protocolos usa (WhatsApp usa XMPP sobre TLS y el protocolo Signal).
3. **Descompilación del APK** Descargar el APK de WhatsApp y descompilarlo con herramientas como jadx o apktool. Esto permite ver el código Java/Kotlin ofuscado, identificar librerías usadas (como libsignal), y descubrir cómo se llaman los métodos de cifrado.

4. **Análisis de Caja Negra** Observar el comportamiento externo: qué pasa si se envía un mensaje sin conexión (se guarda y reenvía), cuántos "ticks" aparecen y cuándo (✓ enviado, ✓✓ entregado, ✓✓ leído), y qué sucede cuando ambos usuarios están offline.
5. **Análisis de Documentación Pública** WhatsApp publicó su whitepaper de seguridad. Este es un recurso legítimo que confirma el uso del Protocolo Signal, las claves de identidad, y el mecanismo de ratcheting de claves.



Resumen del flujo

1. Emisor cifra el mensaje con la clave pública del receptor (Protocolo Signal)
2. El mensaje cifrado viaja por TLS al servidor XMPP — WhatsApp nunca ve el contenido
3. Si el receptor está offline, el mensaje se guarda cifrado en la cola del servidor
4. Los ticks (✓✓) son ACKs: el servidor confirma entrega; el receptor confirma lectura

Fase 4: Prompt para generar la imagen con IA

Prompt:

"Ilustración digital 3D isométrica y detallada, con un estilo cyberpunk tipo plano arquitectónico (blueprint).
Un candado gigante y brillante se encuentra en el centro, partido en dos por la mitad para revelar sus mecanismos internos.

Dentro del candado, se observan flujos de datos visibles etiquetados como 'Cifrado de Extremo a Extremo', 'Protocolo Signal' y 'Servidor XMPP'.

Dos siluetas de smartphones flotan a cada lado, conectadas al candado mediante cables de datos brillantes de color cian.

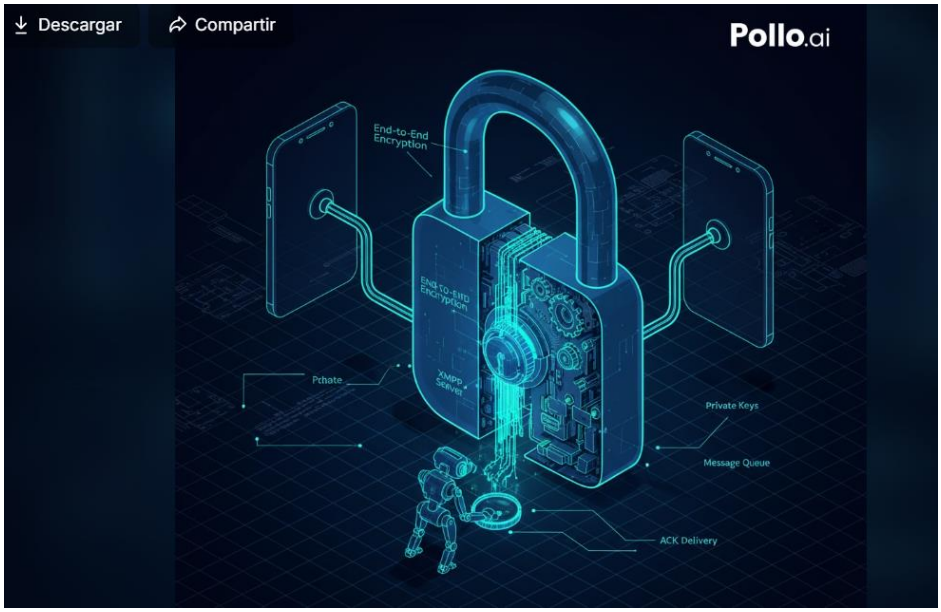
Un pequeño robot investigador se encuentra en la base, escaneando los componentes internos con una lupa láser.

Engranajes y placas de circuito visibles están etiquetados como 'Claves Privadas', 'Cola de Mensajes' y 'Confirmación de Entrega (ACK)'.

Paleta de colores: fondo azul oscuro profundo, detalles en neón cian y verde, líneas de cuadrícula blancas estilo plano arquitectónico, muy detallado, iluminación cinematográfica."

End-to-End Encryption
Signal Protocol
Message Queue
XMPP Server
Private Keys
ACK Delivery
Editar

Descargar
Compartir
Pollo.ai
End-to-End Encryption
Signal Protocol
XMPP Server
Private Keys
ACK Delivery
Message Queue
ACK Delivery



Bibliografía

[1] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016. Disponible en: https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003258/9780137503148

[2] R. S. Pressman y B. R. Maxim, *Ingeniería del software: Un enfoque práctico*, 9.ª ed. México: McGraw-Hill Interamericana, 2021. Disponible en: https://www.vitalsource.com/mx/products/ingenieria-de-software-un-enfoque-practico-roger-s-pressman-bruce-r-v9781456284749

[3] E. J. Chikofsky y J. H. Cross II, “Reverse Engineering and Design Recovery: A Taxonomy,” *IEEE Software*, vol. 7, no. 1, pp. 13–17, Jan. 1990. Disponible en: https://ieeexplore.ieee.org/document/43044

[4] OpenAI, “ChatGPT – Asistente de IA,” Modelo GPT-5.3, 2026. Accedido: abr. 10, 2026. [En línea]. Disponible: https://chat.openai.com